Universidad Autónoma de Nuevo León

Facultad de Ingeniería, Mecánica y Eléctrica

Sistemas Operativos

# Manual de usuario

| Fotografía | Matrícula | Nombre | Carrera | Participación |
|---|---|---|---|---|
| | 2070009 | Cruz Ortega Angely Ximena | ITS | **100%** |
| | 2037907 | García Martínez Ari Sebastián | IAS | **100%** |
| | 2099212 | Pérez Barrón Diego Alonso | ITS | **100%** |

## Dra. Norma Edith Marín Martínez

**Repositorio: ximelovely/PIA_Grupo-015**

**Enero-Junio 2026 San Nicolás de los Garza, NL**

# Tabla de contenido


Tabla de ilustraciones ..... 4

Descripción General del Proyecto ..... 5

Arquitectura del Sistema ..... 5

- Sistema de Diseño Visual ..... 6

Tecnologías y Dependencias ..... 7

- Tecnologías del Frontend ..... 7
- Dependencias Externas ..... 7
- Compatibilidad de Navegadores ..... 7

Análisis de cada pantalla ..... 8

- Barra de Navegación — Componente transversal ..... 8
  - Estructura HTML de la navegación ..... 8
- Página de Inicio — index.html ..... 9
  - Estructura HTML ..... 9
  - Estilos CSS relevantes ..... 10
- AF1 — af1.html ..... 11
  - Estructura HTML ..... 13
- AF2 — af2.html ..... 16
  - Estructura HTML ..... 18
- AF3 — af3.html ..... 19
  - Estructura HTML ..... 20
- AF4 — af4.html ..... 21
  - Estructura HTML ..... 23
- Portafolio de Actividades — actividades.html ..... 25
  - Estructura HTML ..... 26

Simulador de Terminal — terminal.html ..... 27
Estructura HTML del simulador ..... 27
Equipo — equipo.html ..... 31
Estructura HTML ..... 32
Proceso de Despliegue ..... 34
Flujo de despliegue con GitHub Pages ..... 34
Configuración de GitHub Pages ..... 34

# Tabla de ilustraciones

Ilustración 1. Barra de nacegación superior en primer plano, visible en todas las páginas ..... 8
Ilustración 2. Vista completa renderizada de index.html en el navegador ..... 9
Ilustración 3. Parte superior de af1.html ..... 13
Ilustración 4. Tabla comparativa de SO en af1.html completamente visible ..... 14
Ilustración 5. Vista general de af2.html ..... 18
Ilustración 6. Vista completa de af3.html con tipos de memoria y sistemas de archivos ..... 19
Ilustración 7. Parte superior de af4.html ..... 23
Ilustración 8. Vista completa de actividades.html renderizada en el navegador ..... 25
Ilustración 9. Estado inicial de terminal.html con el mensaje de bienvenida y el prompt ..... 27
Ilustración 10. Vista completa de equipo.html mostrando las fotografías y datos de los integrantes ..... 32

Código 1. Estructura HTML de la navegación ..... 8
Código 2. Estructura técnica de la tabla comparativa ..... 14
Código 3. Estructura técnica de las listas ..... 20
Código 4. Estructura HTML del simulador ..... 27
Código 5. Patrón del diccionario de comandos ..... 29
Código 6. Estructura HTML ..... 32
Código 7. Flujo de despliegue con GitHub Pages ..... 34

Tabla 1. Descripción general del proyecto ..... 5
Tabla 2. Arquitectura del sistema ..... 6
Tabla 3. Sistema de diseño visual ..... 6
Tabla 4. Tecnologías del Frontend ..... 7
Tabla 5. Compatibilidad con los navegadores ..... 8

# MANUAL TÉCNICO

## Descripción General del Proyecto

El Portafolio PIA del Grupo 015 es un sitio web estático multi-página desarrollado con HTML5, CSS3 y JavaScript ES6+ puro. Su finalidad es documentar académicamente cuatro actividades formativas sobre Ubuntu 24.04 LTS y proveer un simulador de terminal educativo. El sitio está desplegado en GitHub Pages sin ningún backend ni base de datos.

| Atributo | Valor |
|---|---|
| **Tipo de sistema** | Sitio web estático (sin servidor de aplicaciones) |
| **Lenguajes** | HTML5, CSS3, JavaScript ES6+ |
| **Hosting** | GitHub Pages (sin costo, despliegue automático desde Git) |
| **URL de producción** | https://ximelovely.github.io/PIA_Grupo-015/index.html |
| **Repositorio** | https://github.com/ximelovely/PIA_Grupo-015 |
| **Dependencias externas** | Ninguna (sin CDNs, sin frameworks) |

Tabla 1. Descripción general del proyecto

## Arquitectura del Sistema

El sitio implementa una arquitectura cliente puro (client-side only). No existe procesamiento en el servidor más allá de servir archivos estáticos. El flujo es: GitHub Pages entrega el HTML al navegador del cliente → el navegador interpreta HTML/CSS y ejecuta el JavaScript → toda la lógica e interacción ocurre localmente en el navegador.

| Capa | Tecnología | Responsabilidad |
|---|---|---|
| **Presentación** | HTML5 semántico | Estructura y contenido de cada página |
| **Estilos** | CSS3 (Flexbox/Grid, variables CSS) | Layout responsivo, tipografía, colores, animaciones |

| | | |
|---|---|---|
| **Lógica** | JavaScript ES6+ (Vanilla, sin frameworks) | Simulador de terminal: captura eventos, procesa comandos, actualiza DOM |
| **Hosting** | GitHub Pages (CDN de GitHub) | Entrega de archivos estáticos sobre HTTPS |
| **Control de versiones** | Git + GitHub | Gestión de cambios y despliegue continuo |

**Tabla 2. Arquitectura del sistema**

# Sistema de Diseño Visual

Todos los archivos comparten los mismos tokens de diseño, implementados como CSS interno en cada página:

| Token | Valor | Uso |
|---|---|---|
| **Color primario (crema)** | #F9F6F0 | Fondo general de página |
| **Color secundario (nude)** | #D9C5B3 | Tarjetas, bordes, acentos suaves |
| **Color terciario (salvia)** | #A3B5A6 | Encabezados, botones activos, divisores |
| **Tipografía serif** | Cormorant Garamond | Títulos principales (Google Fonts) |
| **Tipografía sans-serif** | Raleway | Cuerpo de texto, navegación (Google Fonts) |
| **Bordes** | border-radius: 12-16px | Tarjetas y contenedores |

**Tabla 3. Sistema de diseño visual**

# Tecnologías y Dependencias

## Tecnologías del Frontend

| Tecnología | Versión | Rol en el proyecto |
|---|---|---|
| **HTML5** | Estándar W3C | Estructura semántica de todas las páginas |
| **CSS3** | Estándar W3C | Estilos, layout (Flexbox/Grid), animaciones |
| **JavaScript** | ES6+ | Lógica de la terminal simulada (terminal.html) |
| **Google Fonts API** | v2 (CDN) | Carga de Cormorant Garamond y Raleway |

Tabla 4. Tecnologías del Frontend

## Dependencias Externas

El proyecto no tiene dependencias de npm, pip ni ningún gestor de paquetes. Las únicas dependencias remotas son:

- Google Fonts CDN — cargado vía <link rel="stylesheet"> en el <head> de cada página.
- Sin frameworks CSS (Bootstrap, Tailwind, etc.) — todos los estilos son propietarios del proyecto.
- Sin librerías JavaScript externas — la terminal simulada está implementada con JS vanilla.

## Compatibilidad de Navegadores

| Navegador | Soporte | Notas |
|---|---|---|
| **Google Chrome 100+** | Completo | Navegador de referencia para desarrollo |
| **Firefox 95+** | Completo | Sin diferencias visuales significativas |

| | | |
|---|---|---|
| **Safari 15+** | Completo | Verificar fuentes de Google Fonts en offline |
| **Edge 95+** | Completo | Basado en Chromium, sin problemas conocidos |
| **Chrome Mobile** | Completo | Diseño responsivo adaptado |

**Tabla 5. Compatibilidad con los navegadores**

# Análisis de cada pantalla

## Barra de Navegación — Componente transversal

**Ilustración 1. Barra de nacegación superior en primer plano, visible en todas las páginas**

### Estructura HTML de la navegación

```
<!-- Barra de navegación — presente en TODOS los archivos HTML -->
<nav>
  <a href='index.html'>Inicio</a>
  <a href='af1.html'>AF1</a>
  <a href='af2.html'>AF2</a>
  <a href='af3.html'>AF3</a>
  <a href='af4.html'>AF4</a>
  <a href='actividades.html'>Portafolio</a>
  <a href='terminal.html'>Terminal</a>
  <a href='equipo.html'>Equipo</a>
</nav>
```

**Código 1. Estructura HTML de la navegación**

# Página de Inicio — index.html

**Ilustración 2. Vista completa renderizada de index.html en el navegador**

## Estructura HTML

La página index.html está compuesta por tres bloques principales en el body:

- <nav>: Barra de navegación superior con ocho elementos <a> que enlazan al resto de páginas del sitio. Uso de clases CSS para el estilo activo del enlace correspondiente a la página actual.

- <main> o <section> hero: Contiene el <h1> con el título 'Sistemas Operativos' y un párrafo descriptivo del proyecto PIA.
- Tarjetas de acceso rápido: Dos elementos <a> o  con clase tipo card que actúan como botones de acceso a actividades.html y terminal.html, cada uno con un <h3> descriptivo y texto de apoyo.
- <footer>: Pie de página con texto identificativo del grupo y del sistema operativo estudiado.

## Estilos CSS relevantes

- La barra de navegación usa display: flex con justify-content: space-around o similar para distribuir los enlaces horizontalmente.
- Las tarjetas de acceso utilizan border-radius, box-shadow y una transición hover para feedback visual al pasar el cursor.
- El layout general emplea una columna central con max-width para mantener legibilidad en pantallas anchas.

# AF1 — af1.html

Inicio AF1 AF2 AF3 AF4 Portafolio Terminal Equipo

## AF1

## Definición y Componentes del Sistema Operativo - Ubuntu 24.04

### Introducción

Ubuntu es un sistema operativo de código abierto basado en la distribución Debian de GNU/Linux. Fue lanzado en 2004 por la empresa Canonical, liderada por Mark Shuttleworth, con el objetivo de hacer que Linux fuera accesible y fácil de usar para el usuario promedio sin perder la potencia que requieren los desarrolladores y administradores de sistemas. La filosofía de Ubuntu se basa en el término africano homónimo, que se traduce aproximadamente como "humanidad hacia otros". Ubuntu se destaca por su ciclo de lanzamientos predecible (versiones regulares cada seis meses y versiones LTS o Long Term Support cada dos años), su robusta seguridad, su extensa comunidad de soporte y su naturaleza gratuita.

*Figura 1.1: Diagrama de arquitectura del kernel Linux monolítico modular.*

## Componentes del Sistema Operativo y sus funciones

**1. El Núcleo o Kernel**

El kernel de Ubuntu es de tipo monolítico modular. Maneja funciones críticas y carga "módulos" bajo demanda.

- **Gestión de Interrupciones:** Maneja señales del hardware y decide qué proceso atenderlas.
- **Abstracción de Hardware:** Interfaz uniforme para programas.

**2. Gestión de Procesos**

Ubuntu utiliza el Completely Fair Scheduler (CFS) para maximizar la eficiencia.

- **Creación y Terminación:** Llamadas como fork() y exec().
- **Sincronización:** Semáforos y bloqueos (mutex).
- **Estados:** "Ejecutándose", "Durmiendo" o "Zombi".

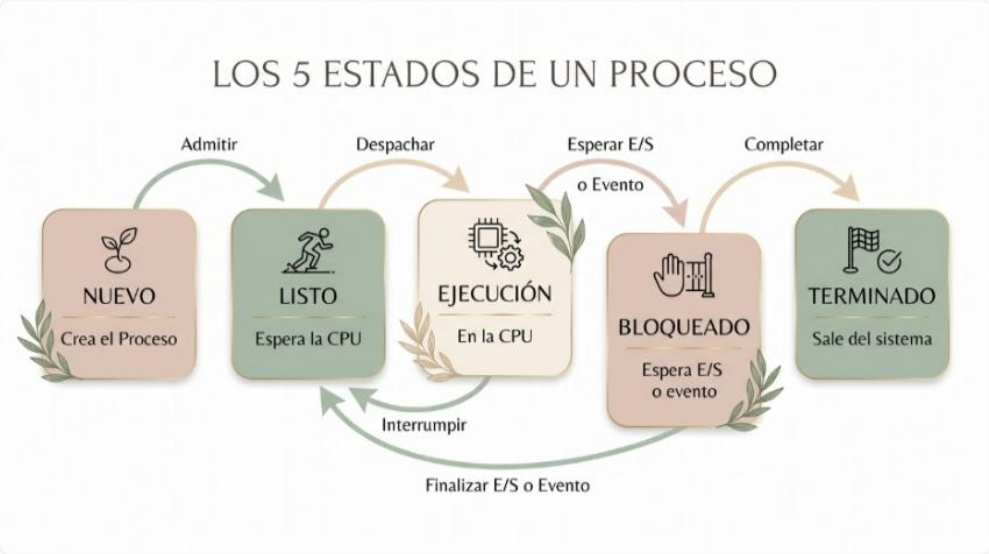



*Figura 1.2: Diagrama de los diferentes estados de un proceso en Ubuntu.*

**3. Gestión de Memoria Principal**

Modelo de Memoria Virtual mediante paginación (4KB). Asegura protección entre programas y gestiona el Swap.

**4. Gestión del Sistema de Archivos**

Uso predeterminado de ext4 con jerarquía unificada desde la raíz /, soportado por Inodos y Journaling para recuperación.

**5. Gestión de Entrada/Salida (I/O)**

Drivers y áreas de Buffering/Spooling para compensar velocidades entre CPU y periféricos.

**6. Sistema de Seguridad**

Módulos como AppArmor (MAC) y Control de Acceso Discrecional (DAC) para permisos de usuarios.

## Función y características en el manejo de recursos

- **Gestión del Procesador (CPU):** Ubuntu utiliza CFS con planificación basada en tiempo de ejecución usando un árbol rojo-negro. Permite ajustar prioridades mediante valores "nice".
- **Gestión de la Memoria (RAM):** Memoria virtual, paginación bajo demanda, OOM Killer que sacrifica procesos cuando la memoria se agota.
- **Gestión de Dispositivos (I/O):** Sistemas de archivos virtuales (VFS), manejo de interrupciones, DMA para transferencias directas.
- **Almacenamiento Secundario:** Planificadores de disco (Deadline, BFQ, Kyber), caché de escritura, journaling con ext4.

## Clasificación según el número de usuarios

**Multiusuario:** Ubuntu es estrictamente multiusuario. Varios usuarios pueden iniciar sesión al mismo tiempo (local o vía SSH), con aislamiento de archivos en /home/, y solo el superusuario (sudo) puede hacer cambios globales.

## Partes de gestión de un sistema operativo

- **Gestión de Arranque (systemd):** Primer proceso (PID 1). Usa systemctl.
- **Gestión de Paquetes:** APT para .deb, Snap para contenedores.
- **Gestión de Redes:** NetworkManager (escritorio) y Netplan (servidores).
- **Autenticación:** PAM (Módulos de autenticación conectables).
- **Volúmenes Lógicos (LVM):** Agrupa y expande particiones.
- **Registros:** journald y Syslog.

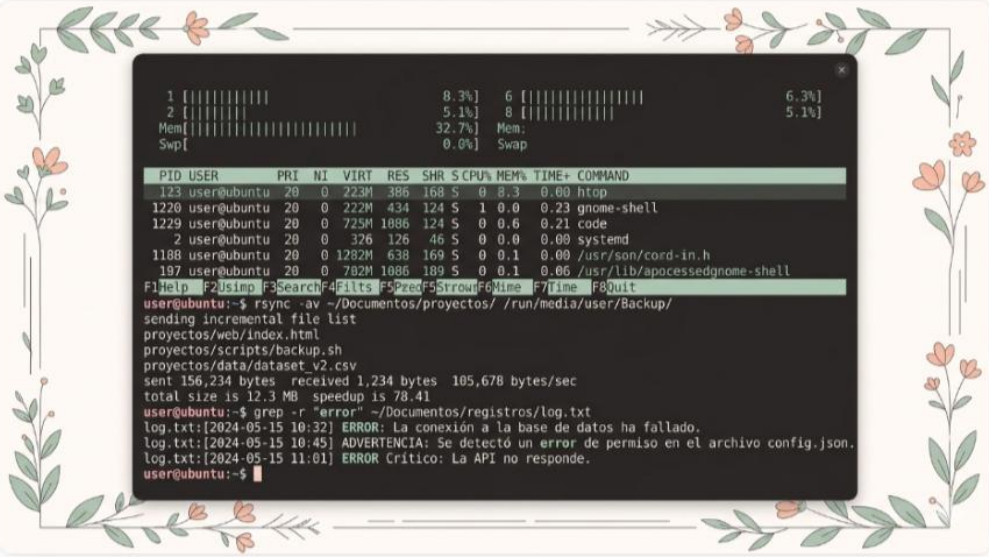



*Figura 1.3: Utilerías de terminal comunes en la gestión de Ubuntu.*

**Ilustración 3. Parte superior de af1.html**

# Estructura HTML

- <h1>: Título de la actividad 'AF1'.

- <h2>: Subtítulo 'Definición y Componentes del Sistema Operativo - Ubuntu 24.04'.
- Secciones de contenido (<section> o ): Cada subtema (kernel, procesos, memoria, I/O, seguridad) se divide en subsecciones con <h3> o <h4>.
- <img> con src relativo: Tres imágenes de diagrama — imagen-af1-kernel.jpg, imagen-af1-estados-proceso.jpg.png e imagen-af1-utilities.jpg.png. Cada imagen va seguida de un elemento <p> o <figcaption> con el número y descripción de la figura.
- <table>: Tabla HTML semántica al final de la página para el cuadro comparativo de sistemas operativos, con <thead>, <tbody>, <tr>, <th> y <td>.

Cuadro comparativo de SO actuales

| Sistema Operativo | Tipo / Núcleo | Licencia | Entorno Principal |
|---|---|---|---|
| Ubuntu Desktop | Monolítico (Linux) | Open Source | Escritorio |
| Windows 11 | Híbrido (NT) | Propietario | Escritorio |
| macOS | Híbrido (XNU/Darwin) | Propietario | Escritorio |
| Ubuntu Touch | Monolítico (Linux) | Open Source | Móvil |
| iOS | Híbrido (XNU/Darwin) | Propietario | Móvil |
| Ubuntu Server | Monolítico (Linux) | Open Source | Servidor (CLI) |
| Windows Server | Híbrido (NT) | Propietario | Servidor |

Grupo 015 • Ubuntu 24.04 LTS

**Ilustración 4. Tabla comparativa de SO en af1.html completamente visible**

## Estructura técnica de la tabla comparativa

**Código 2. Estructura técnica de la tabla comparativa**

```
<table>
  <thead>
    <tr>
      <th>Sistema Operativo</th>
```

```
      <th>Tipo / Núcleo</th>
      <th>Licencia</th>
      <th>Entorno Principal</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>Ubuntu Desktop</td><td>Monolítico (Linux)</td>
        <td>Open Source</td><td>Escritorio</td></tr>
    <!-- ...más filas... -->
  </tbody>
</table>
```

# AF2 — af2.html

Inicio AF1 AF2 AF3 AF4 Portafolio Terminal Equipo

## AF2

## Procesos, Hilos y Concurrencia en Ubuntu 24.04

### Problemas Centrales de la Concurrencia

La concurrencia surge cuando dos o más procesos o hilos se ejecutan de manera simultánea y comparten recursos del sistema. Esto permite un mejor aprovechamiento del procesador pero introduce problemas complejos que deben ser controlados.

*Figura 2.1: Esquema visual de una Condición de Carrera.*

### Condiciones de Carrera (Race Conditions)

Una condición de carrera ocurre cuando el resultado final de una ejecución depende del orden en que se intercalan las instrucciones de los procesos. Este orden depende del planificador del sistema operativo y de factores como interrupciones o tiempos de ejecución.

Comando útil en Ubuntu para detectar condiciones de carrera:

```
$ strace -p PID
```

*Figura 2.2: Implementación de la Exclusión Mutua en una Sección Crítica.*

## Secciones Críticas

Una sección crítica es la parte del código donde un proceso accede a un recurso compartido que no puede ser utilizado simultáneamente por otro proceso sin provocar inconsistencias.

Ejemplos de operaciones en sección crítica:

- Actualizar una variable compartida.
- Escribir en un archivo común.
- Modificar estructuras del sistema operativo.

### Requerimientos para una Solución

- **Exclusión Mutua:** Garantiza que solo un proceso puede estar dentro de su sección crítica en un momento dado.
- **Progreso:** Si ningún proceso está ejecutando su sección crítica, la decisión de quién entra no puede posponerse indefinidamente.
- **Espera Limitada:** Asegura que existe un límite de tiempo para que un proceso acceda para que no haya una inanición (starvation).

**Ilustración 5. Vista general de af2.html**

## Estructura HTML

- <h1> 'AF2' y <h2> 'Procesos, Hilos y Concurrencia en Ubuntu 24.04': Encabezados de la página.
- Secciones de contenido: Tres bloques principales — Condiciones de Carrera, Secciones Críticas y Comandos de gestión — cada uno con su correspondiente <h3> o <h4>.
- Tres elementos <img>: imagen-af2-race-condition.jpg.png, imagen-af2-seccion-critica.jpg.png e imagen-af2-hilos.jpg.png. Las rutas incluyen el subdirectorio 'Video e imágenes/' con espacio en el nombre, lo cual en URLs debe estar codificado como %20 o entre comillas en el HTML.
- Bloque de código de comandos: Elemento <pre><code> o similar que contiene los comandos bash (ps aux, htop, kill, nice, taskset) con formato monoespaciado.

# AF3 — af3.html

Inicio AF1 AF2 AF3 AF4 Portafolio Terminal Equipo

## AF3

### Memoria y Sistemas de Archivos

La forma en que Ubuntu recuerda y organiza nuestra información es vital para su elegancia y velocidad.

**Tipos de Memoria**

- **RAM:** Memoria principal, ultrarrápida pero volátil.
- **Swap (Intercambio):** Espacio en disco que Ubuntu usa como "RAM de emergencia" (típicamente `~/swapfile`).
- **Caché y Buffer:** RAM sobrante usada inteligentemente para guardar archivos recientes y acelerar el sistema.

**Sistemas de Archivos**

- **ext4:** El estándar confiable por defecto en Ubuntu.
- **Btrfs / XFS:** Para usuarios avanzados o servidores (excelentes para instantáneas).
- **NTFS / FAT:** Sistemas de Windows que Ubuntu puede leer sin problemas.

### Control de Espacios en Memoria

¿Cómo sabemos cuánta memoria nos queda? Usa estos comandos:

```
$ free -h
$ cat /proc/meminfo
$ vmstat 2
```

### Problemas Frecuentes y Soluciones Prácticas

- **Fragmentación:** Espacios vacíos desperdiciados. En Linux (ext4) la fragmentación externa es casi inexistente, ¡no necesitas desfragmentar!
- **Fugas de Memoria (Memory Leaks):** Un programa que no devuelve la RAM. Identifícalo con `top` y ciérralo con `kill`.
- **Sobrecarga de Caché:** Si necesitas limpiarla manualmente:

  ```
  # echo 3 > /proc/sys/vm/drop_caches
  ```

- **Swapping excesivo (Thrashing):** Cuando la PC usa el disco duro constantemente por falta de RAM. *Solución:* Ajustar la swappiness.

  ```
  $ sysctl vm.swappiness=10
  ```

Grupo 015 • Ubuntu 24.04 LTS

**Ilustración 6. Vista completa de af3.html con tipos de memoria y sistemas de archivos**

## Estructura HTML

- <h1> 'AF3' y <h2> 'Memoria y Sistemas de Archivos': Encabezados principales de la página.
- Listas <ul> con <li>: Se usan para los tipos de memoria (RAM, Swap, Caché) y para los sistemas de archivos (ext4, Btrfs/XFS, NTFS/FAT), aprovechando el marcado semántico de HTML5.
- Sección de comandos: Bloque <pre> o <code> con los tres comandos de control de memoria (free -h, cat /proc/meminfo, vmstat 2).
- Sección de problemas y soluciones: Lista descriptiva con cuatro problemas comunes, cada uno con su solución en línea de código.

## Estructura técnica de las listas

**Código 3. Estructura técnica de las listas**

```
<ul>
  <li><strong>RAM:</strong> Memoria principal, ultrarrápida pero volátil.</li>
  <li><strong>Swap (Intercambio):</strong> Espacio en disco como 'RAM de emergencia'
      (típicamente /swapfile).</li>
  <li><strong>Caché y Buffer:</strong> RAM sobrante usada para acelerar el sistema.</li>
</ul>
```

# AF4 — af4.html

Inicio AF1 AF2 AF3 AF4 Portafolio Terminal Equipo

## AF4

## Seguridad en Sistemas Operativos - Implementación en Ubuntu

### 1. Identificación de Amenazas

Comandos utilizados en Ubuntu para identificar y auditar el estado del sistema:

```
$ top # Monitoreo de procesos en tiempo real
$ netstat -tulnp # Ver puertos abiertos y servicios escuchando
$ who # Usuarios conectados al sistema
$ last # Historial de inicios de sesión
$ journalctl -xe # Revisar logs del sistema
```

#### 2. Seguridad a nivel sistema

```
$ sudo adduser usuario_seguro
$ sudo chmod 700
/home/usuario_seguro
```

#### 3. Firewall (UFW)

```
$ sudo ufw enable
$ sudo ufw allow 22/tcp
$ sudo ufw allow 80/tcp
$ sudo ufw allow 443/tcp
$ sudo ufw status verbose
```

```
pi@pimylifeup:~ $ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
pi@pimylifeup:~ $ sudo ufw status
Status: active

To                         Action      From
--                         ------      ----
80                         ALLOW       Anywhere
443/tcp                    ALLOW       Anywhere
Anywhere                   ALLOW       192.168.0.1
3306/tcp                   ALLOW       192.168.0.2
22565                      DENY        Anywhere
3478/tcp                   DENY        Anywhere
23/tcp                     DENY        Anywhere
Anywhere                   DENY        192.168.0.3
22                         DENY        192.168.0.4
22                         LIMIT       Anywhere
22/tcp                     LIMIT       Anywhere
80 (v6)                    ALLOW       Anywhere (v6)
443/tcp (v6)               ALLOW       Anywhere (v6)
22565 (v6)                 DENY        Anywhere (v6)
3478/tcp (v6)              DENY        Anywhere (v6)
23/tcp (v6)                DENY        Anywhere (v6)
22 (v6)                    LIMIT       Anywhere (v6)
22/tcp (v6)                LIMIT       Anywhere (v6)
```

PiMyLifeUp

*Figura 4.1: Reglas de red configuradas en el firewall UFW.*

## 4. Protección contra ataques de fuerza bruta (Fail2ban)

```
$ sudo apt update
$ sudo apt install fail2ban
$ sudo systemctl start fail2ban
$ sudo systemctl enable fail2ban
$ sudo fail2ban-client status sshd
```

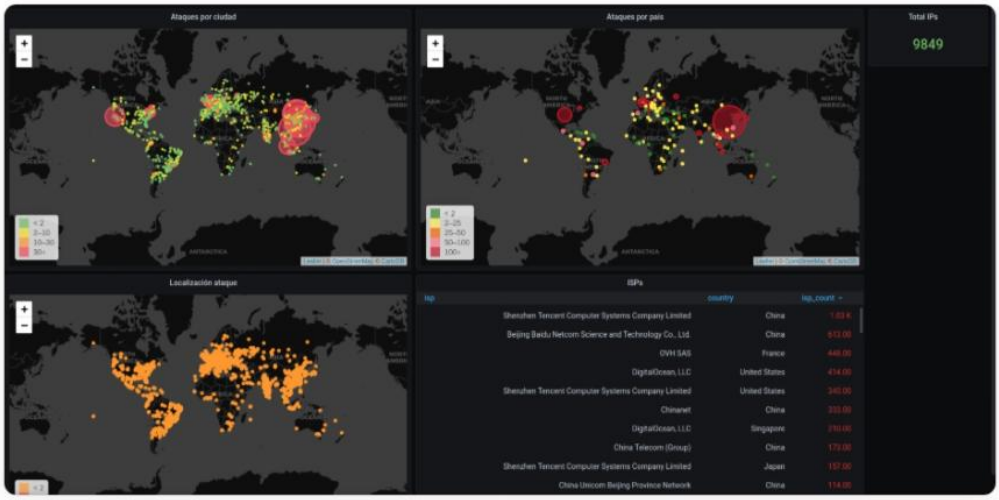



*Figura 4.2: Protección activa de puertos mediada por Fail2ban.*

## 5. Backups y recuperación

```
# Copia simple
$ cp -r /home/usuario /home/backup_usuario

# Copia incremental con rsync
$ rsync -av --delete /home/usuario /media/backup/

# Respaldo con compresión
$ tar -czvf backup_$(date +%Y%m%d).tar.gz /home/usuario
```

*Figura 4.3: Arquitectura de respaldo basada en el esquema 3-2-1.*

6. Cifrado de archivos y discos

```
# Cifrado de archivo individual con GPG
$ gpg -c archivo_sensible.txt

# Descifrar
$ gpg archivo_sensible.txt.gpg

# Verificar si LUKS (cifrado de disco) está activo:
$ lsblk -o NAME,FSTYPE,MOUNTPOINT | grep crypto
```

Herramientas Adicionales

Monitoreo y Auditoría

**Lynis:** `sudo lynis audit system`

**Tcpdump:** `sudo tcpdump -i eth0 -c 100 -w cap.pcap`

**Auditd:** Auditoría detallada del sistema.

Protección Especializada

**ClamAV:** Antivirus para Linux.

**Rkhunter:** Detección de rootkits.

**AppArmor:** Control de acceso obligatorio (MAC).

10. Conclusión

La implementación de herramientas de seguridad en Ubuntu permite aplicar de manera práctica los conceptos teóricos de protección en sistemas operativos. Un sistema Ubuntu bien configurado (con firewall, fail2ban, backups cifrados y monitoreo) es altamente resistente a amenazas externas.

Grupo 015 • Ubuntu 24.04 LTS

**Ilustración 7. Parte superior de af4.html**

## Estructura HTML

af4.html es la página más extensa y con mayor densidad de bloques de código. Su estructura sigue una numeración de secciones del 1 al 10:

- Secciones 1–6: Bloques <h3> o <h4> numerados con su contenido en listas <ul> y/o elementos <pre><code> para los bloques de comandos bash.

- Tres elementos <img>: imagen-af4-firewall.jpg, imagen-af4-fail2ban.jpg e imagen-af4-backup.jpg. A diferencia de las imágenes de AF1 y AF2, estas están en la raíz del proyecto (no en el subdirectorio 'Video e imágenes/').
- Sección de herramientas adicionales (sección implícita): Lista con Lynis, Tcpdump, Auditd, ClamAV, Rkhunter y AppArmor, cada uno con su comando de uso en línea.
- Conclusión (sección 10): Párrafo <p> final con la síntesis del aprendizaje sobre seguridad en Ubuntu.

# Portafolio de Actividades — actividades.html

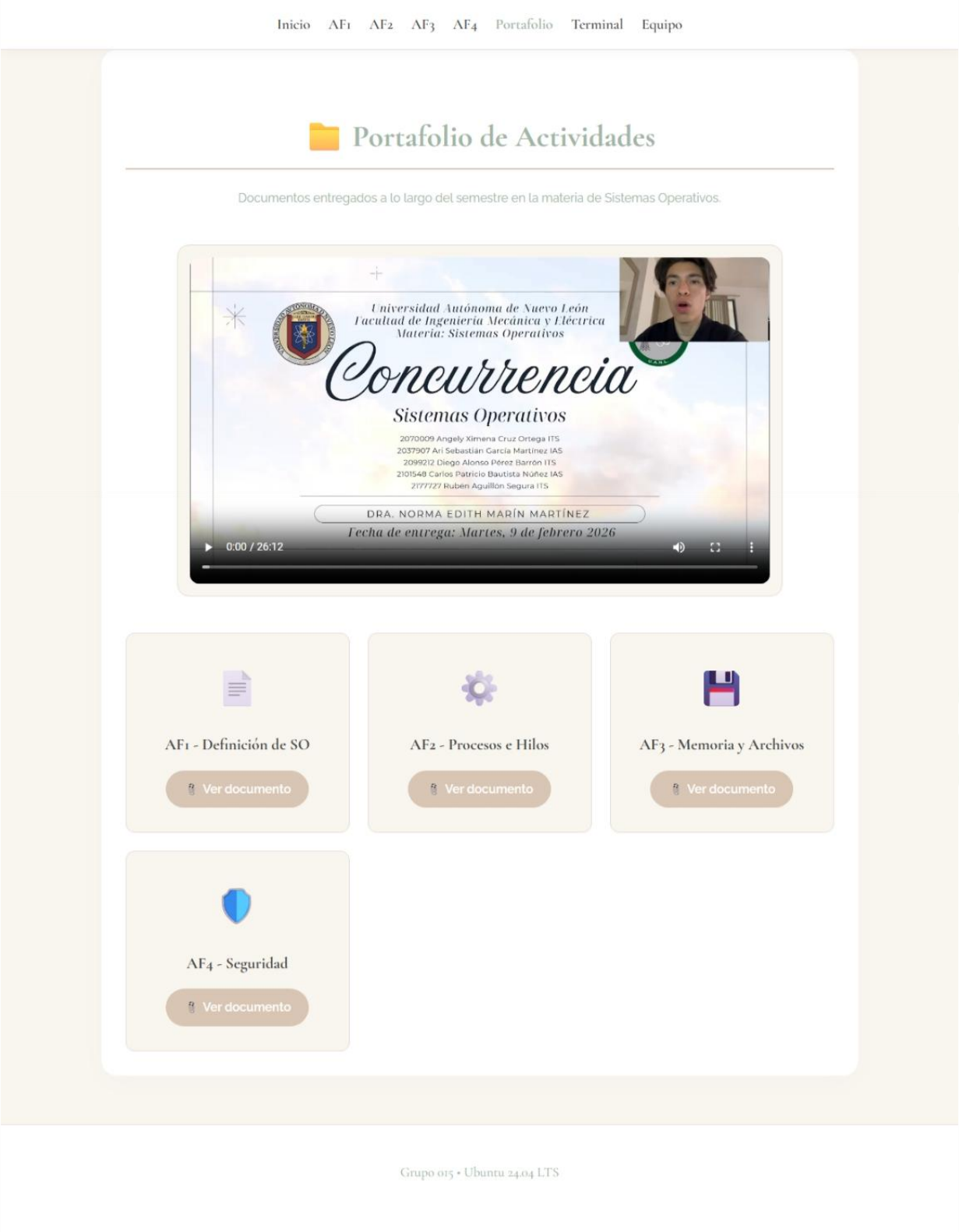



**Ilustración 8. Vista completa de actividades.html renderizada en el navegador**

## Estructura HTML

actividades.html actúa como índice del portafolio del semestre. Su estructura es principalmente de listas o tarjetas que referencian las actividades formativas:

- Encabezado: <h1> o <h2> identificativo de la sección.
- Listado de actividades: Elementos de lista o tarjetas  con clase card o similar, con el nombre de cada actividad y enlace a su página correspondiente.
- La página no requiere JavaScript para su funcionamiento.

# Simulador de Terminal — terminal.html

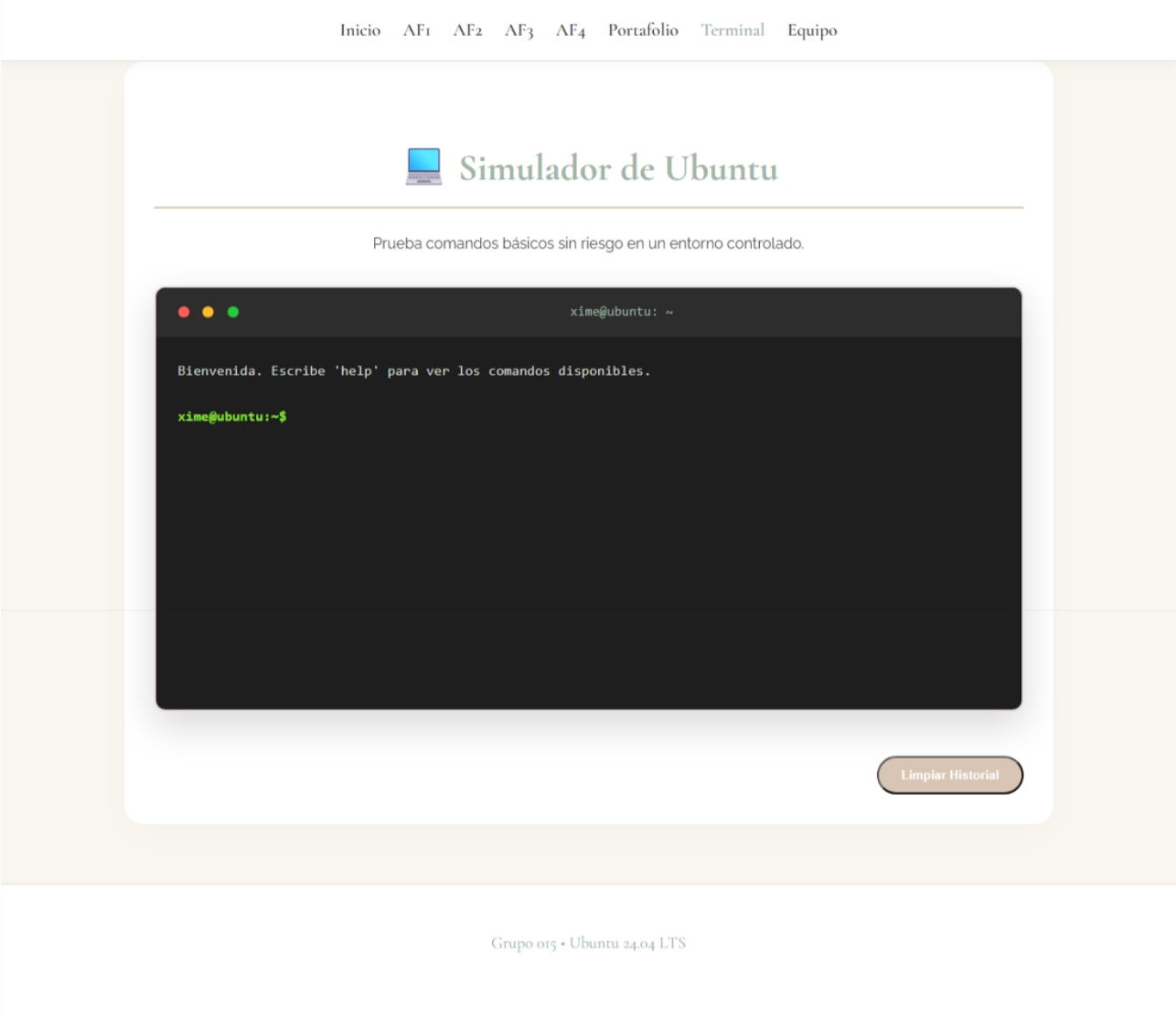



**Ilustración 9. Estado inicial de terminal.html con el mensaje de bienvenida y el prompt**

## Estructura HTML del simulador

terminal.html es la única página del sitio con JavaScript funcional. Su estructura HTML crea la interfaz visual de la terminal:

**Código 4. Estructura HTML del simulador**

```
<!-- Estructura base del simulador -->
<div class='terminal-container'>
  <div class='terminal-header'>
```

```
    <span>xime@ubuntu: ~</span>
  </div>

  <div id='output' class='terminal-output'>
    <!-- Aquí se inyecta el historial de comandos via JS -->
    <p>Bienvenida. Escribe 'help' para ver los comandos disponibles.</p>
  </div>

  <div class='terminal-input-line'>
    <span class='prompt'>xime@ubuntu:~$</span>
    <input type='text' id='input' autofocus
           placeholder='Escribe un comando...' />
  </div>
</div>

<button onclick='clearHistory()'>Limpiar Historial</button>
```

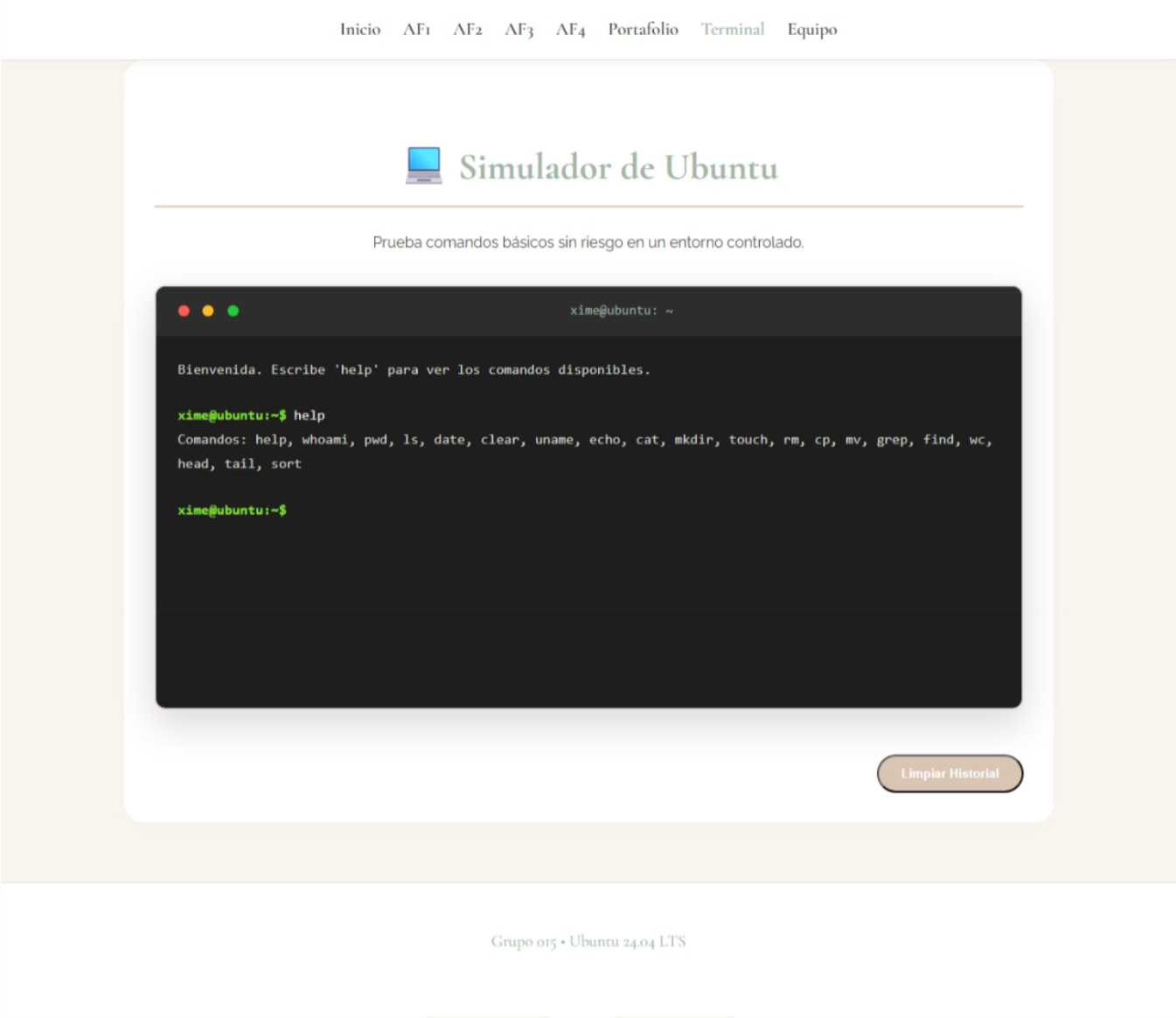
## Lógica JavaScript del simulador

Toda la lógica del simulador está embebida dentro de un bloque <script> al final de terminal.html. El flujo de ejecución es el siguiente:

- Escucha de eventos: Se registra un event listener en el elemento <input> para el evento keydown. Cuando la tecla presionada es Enter (key === 'Enter' o keyCode === 13), se activa la función de procesamiento.
- Captura del comando: Se lee el valor actual del campo de texto (input.value.trim()) y se convierte a minúsculas para normalizar la entrada.
- Lookup en el diccionario de comandos: El comando normalizado se busca en un objeto JavaScript que mapea nombres de comandos a sus respuestas predefinidas (strings o funciones).
- Generación de respuesta: Si el comando existe, se obtiene su respuesta. Si no existe, se retorna un mensaje de error genérico del tipo 'Comando no encontrado'.
- Actualización del DOM: Se crea un nuevo elemento <p> o  con el texto del prompt + el comando escrito, y otro con la respuesta. Ambos se agregan (appendChild) al div #output.
- Scroll automático: Después de agregar los nuevos elementos, se ejecuta output.scrollTop = output.scrollHeight para mantener visible el final del historial.

- Limpieza del input: Se vacía el campo de texto (input.value = '') para listo para el siguiente comando.

## Patrón del diccionario de comandos




**Código 5. Patrón del diccionario de comandos**

```
// Estructura del diccionario de comandos en terminal.html
const commands = {
  'help': 'Comandos disponibles: ls, pwd, whoami, date, echo...',
  'ls':   'Desktop  Documents  Downloads  Music  Pictures',
  'pwd':  '/home/xime',
  'whoami': 'xime',
  'date': () => new Date().toString(),  // Respuesta dinámica
```

```
  'clear': () => { clearHistory(); return ''; },
  // ...más comandos...
};

// Función principal de procesamiento
input.addEventListener('keydown', function(e) {
  if (e.key === 'Enter') {
    const cmd = input.value.trim().toLowerCase();
    const response = typeof commands[cmd] === 'function'
                     ? commands[cmd]()
                     : (commands[cmd] || `${cmd}: command not found`);
    appendToOutput(cmd, response);
    input.value = '';
  }
});
```

# Equipo — equipo.html

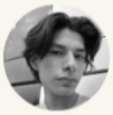
Inicio AF1 AF2 AF3 AF4 Portafolio Terminal Equipo

## Equipo de Trabajo

**Angely Ximena Cruz Ortega**

2070009

Mi conclusión para este proyecto puedo decir que aprendí mucho, especialmente en cuanto a la implementación de medidas de seguridad en sistemas operativos. desde siempre el tema de esta materia ha sido algo que me ha interesado profundamente, por lo que poder aplicar estos conocimientos en un sistema operativos que, pese a no ser mi favorito, me agrada, ha sido increíble.



**García Martínez Ari Sebastián**

2037907

Como estudiante, considero que Ubuntu es la alternativa ideal frente a sistemas tradicionales. Su mayor ventaja es que es completamente gratuito y ligero, lo que ayuda a que nuestras laptops no se traben tanto al investigar o programar. A diferencia de otros sistemas, Linux te da el control total: es seguro, no te fuerza a actualizaciones molestas y tiene una comunidad enorme que siempre ayuda si algo falla. En resumen, usar Ubuntu nos permite aprender cómo funciona realmente nuestra tecnología sin gastar en licencias costosas.

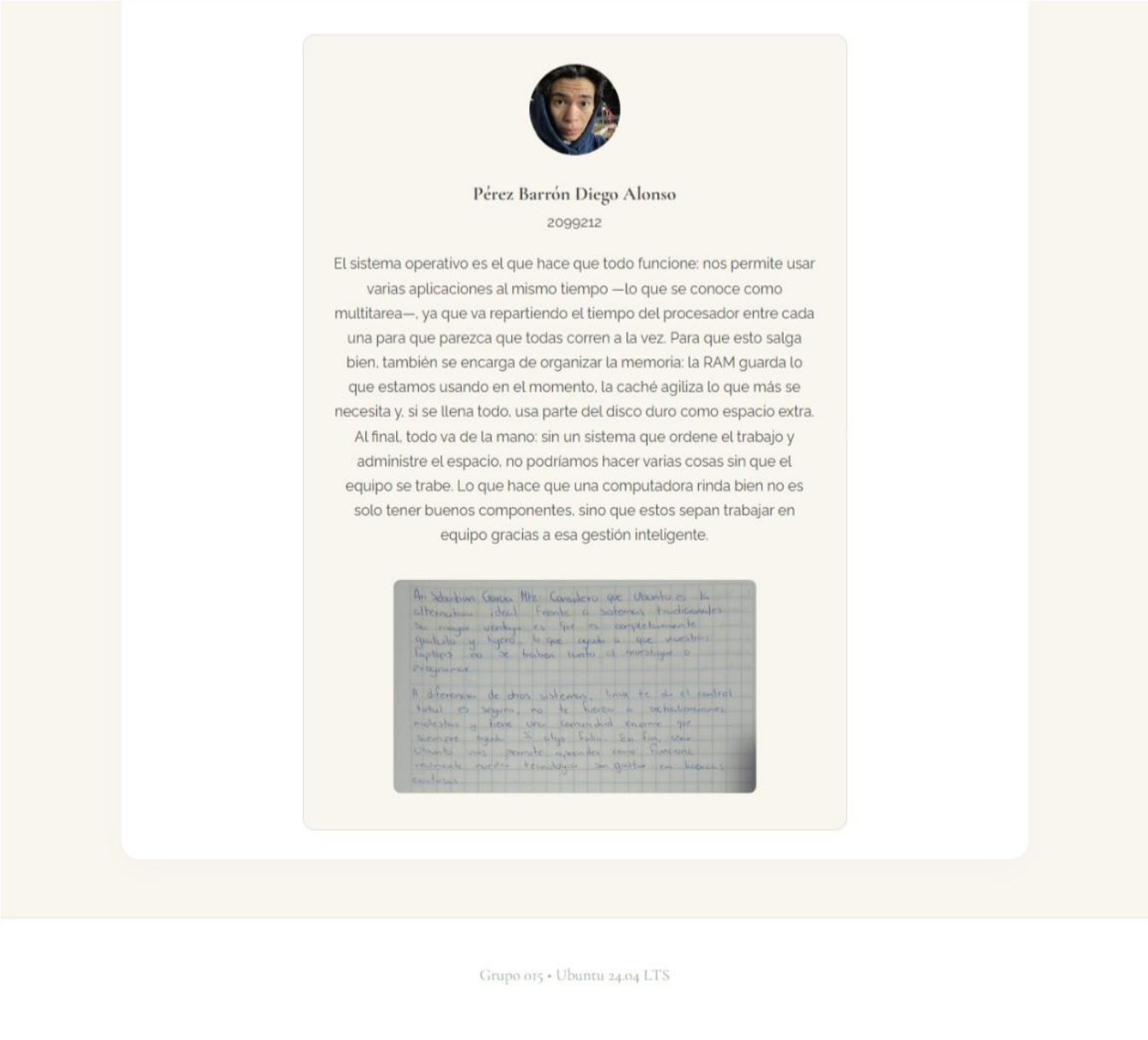



**Ilustración 10. Vista completa de equipo.html mostrando las fotografías y datos de los integrantes**

# Estructura HTML

equipo.html presenta a los integrantes del Grupo 015. Su estructura HTML es repetitiva por cada integrante y sigue un patrón de tarjeta de perfil:

**Código 6. Estructura HTML**

```
<!-- Estructura de tarjeta de integrante (se repite por cada uno) -->
<div class='team-member'>
  <img src='Video e imágenes/Integrante 1.jpg'
       alt='Foto de Integrante 1'>
```

```
  <h4>Angely Ximena Cruz Ortega</h4>
  <p>Matrícula: 2070009</p>
  <p>Mi conclusión para este proyecto...</p>
</div>
```

# Proceso de Despliegue

## Flujo de despliegue con GitHub Pages

**Código 7. Flujo de despliegue con GitHub Pages**

```
# Clonar el repositorio localmente
git clone https://github.com/ximelovely/PIA_Grupo-015.git
cd PIA_Grupo-015

# Hacer cambios en los archivos del sitio
# (editar HTML/CSS/JS con cualquier editor de texto)

# Agregar los cambios al stage de Git
git add .

# Registrar el commit
git commit -m "Descripción del cambio realizado"

# Subir al repositorio remoto — activa el despliegue automático
git push origin main

# El sitio se actualiza en GitHub Pages en 1-5 minutos
```

### Configuración de GitHub Pages

El despliegue se configura una sola vez en: Settings del repositorio > Pages > Branch: main > / (root). Una vez configurado, cualquier push a la rama main actualiza el sitio automáticamente.